

to the virtual machines. We can share the Internet connection available on the host machine with the virtual machine using an interface argument.

Use the `--eth0=tap.TAP-ADDRESS,GUEST-ADDRESS` argument along with the `vstart` command to share the Internet connection. `TAP-ADDRESS` is the address attached with `eth0` and `GUEST-ADDRESS` is the address attached for a new virtual interface created in the host machine.

The following example shows how to share an Internet connection with the virtual machine:

```
slynnux@gnubox:~$ vstart pc1 --eth0=tap.192.168.1.98 --eth1=A
```

Now you can *ping* from `pc1` to your Internet gateway to check the connectivity.

Set *nameserver*, by executing the following command:

```
pc1# echo nameserver 192.168.0.1 > /etc/resolvconf // 192.168.0.1 is the nameserver here.
pc1# apt-get update
```

You can now install additional applications to the virtual machine by using *apt-get*.

```
pc1# apt-get install <package name>
```

Netkit Lab

Manually running `vstart` and a series of commands like *route*, *ifconfig*, etc., becomes very difficult and complex in the case of emulating comparatively large networks. It also requires doing the same thing again and again if we are to emulate the same network topology. Netkit Lab, an add-on script, is an exciting feature of the Netkit toolkit. You can write Netkit Lab configuration files such that by simply executing Netkit Lab, we can configure emulation on the network. Numerous virtual machines can be initialised from Netkit Lab configuration files.

Let us see how to make Netkit Lab work.

Create a directory (say, *netkit_testlab*) and include the following files in the *netkit_testlab* directory.

- *lab.conf* – this file consists of configuration details about the virtual machines like interfaces and collision domains.
- *<virtual_machine>.startup* – this file consists of commands to be executed initially while the virtual machine starts.
- *<virtual_machine>* – a blank directory with the same name as that of the virtual machine.

Let us set up the Netkit Lab to bridge two machines through a router with different IP address families:

```
slynnux@gnubox:~$ mkdir netkit_testlab
```

```
slynnux@gnubox:~$ cd netkit_testlab
slynnux@gnubox:~/netkit_testlab$ mkdir pc1 pc2 r1
slynnux@gnubox:~/netkit_testlab$ vim lab.conf
```

Type the following lines excluding the comments starting with `//` and save the *lab.conf* text file:

```
r1[0]="A" // specifies eth0, collision domain = A
r1[1]="B" // specifies eth1, collision domain=B

pc1[0]="A" // specifies eth0, collision domain=A
pc2[0]="B" // specifies eth0, collision domain=B
```

Type the commands to be executed during start up and save to a text file:

```
slynnux@gnubox:~/netkit_testlab$ vim pc1.startup
ifconfig eth0 192.168.0.2
route add default gw 192.168.0.1
```

```
slynnux@gnubox:~/netkit_testlab$ vim pc2.startup
ifconfig eth0 192.168.1.2
route add default gw 192.168.1.1
```

```
slynnux@gnubox:~/netkit_testlab$ vim r1.startup
ifconfig eth0 192.168.0.1
ifconfig eth1 192.168.1.1
```

Now let's start the Netkit Lab using the *lstart* command:

```
slynnux@gnubox:~$ cd netkit_testlab // change current directory to the lab directory
slynnux@gnubox:~/netkit_testlab$ lstart
```

In order to shut down all the machines, use the *lhalt* command from the Netkit Lab directory:

```
slynnux@gnubox:~/netkit_testlab$ lhalt
```

Try writing the *lab.conf* for more complex network topologies.

Netkit is arguably the best way to start experimenting with networking. Some universities are already using Netkit as a teaching aid. Through the UML-supported backbone, Netkit provides a real-time experience on emulation on network topologies. Find more bytes from www.netkit.org. Happy hacking till we meet again! 

By: Sarath Lakshman

The author is a Hacktivist of Free and Open Source Software from Kerala. He loves working on the GNU/Linux environment and contributes to the PitiVi video editor project. He is also the developer of SLYNIX, a distro for newbies. He blogs at www.sarathlakshman.info